

CS-1201

OBJECT ORIENTED PROGRAMMING

Lab

By

Engr. Muhammad Awais

Feb 24, 2026

Lab Contents

- Structures
- Arrays of Structures
- Structure Pointers

Lab Objectives

The objective of this lab is to apply the:

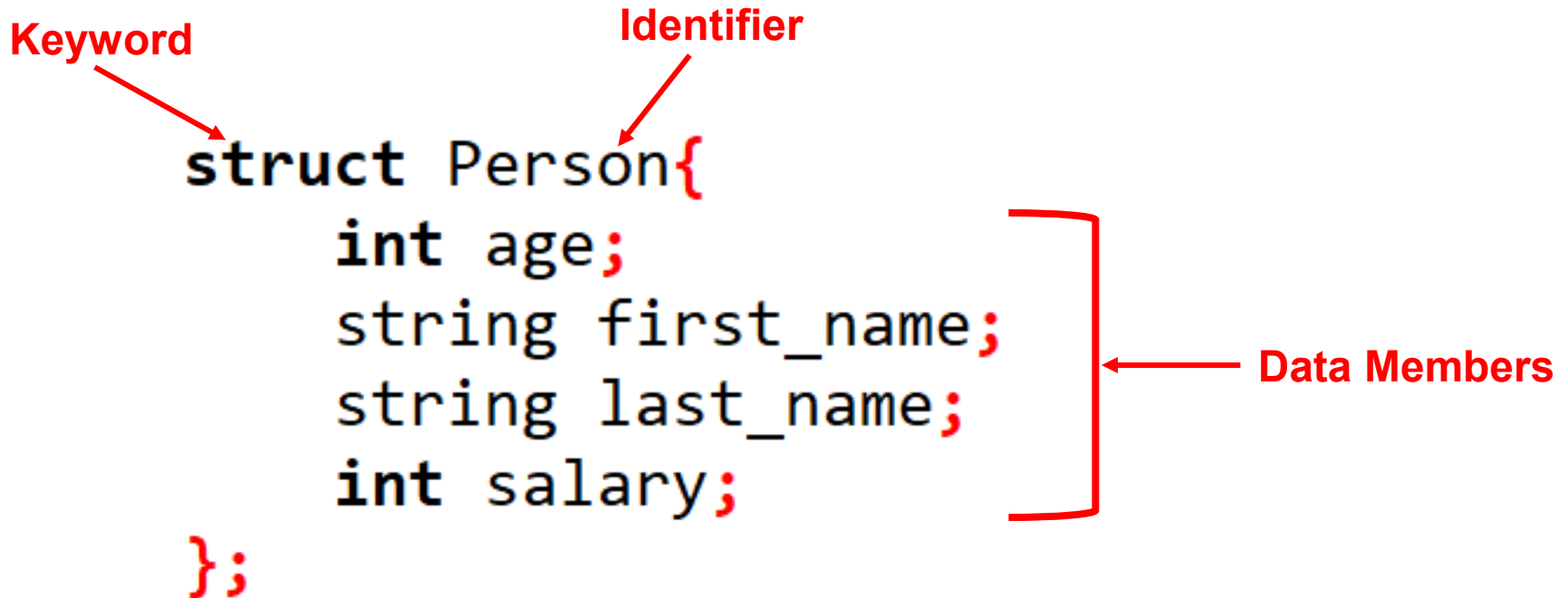
- concept of structures in C++.
- concept of arrays of structures.
- concept of structure pointers.

Structures

- A structure is a collection of **variables of different data types** and **member functions** under a single name.
- Suppose you want to store some information about a person: their **first_name**, **last_name**, **age**, and **salary**. You can easily create different variables—**first_name**, **last_name**, **age**, **salary** to store this information separately.
- For multiple people, you'd need **first_name1**, **last_name1**, **age1**, **salary1**, **first_name2**, **last_name2**, **age2**, **salary2**, etc. The number of variables grows quickly, making the code messy.
- A better approach is to have a collection of all related information under a single name, such as **Person** and use it for every individual.

How to Declare a Structure in C++

- A structure is a user-defined data type in C/C++. A structure creates a data type that can be used to group **items of possibly different types** into a single type.



The diagram illustrates the components of a C++ structure declaration. The code is as follows:

```
struct Person{  
    int age;  
    string first_name;  
    string last_name;  
    int salary;  
};
```

Annotations with red arrows and text:

- Keyword**: Points to the word `struct`.
- Identifier**: Points to the word `Person`.
- Data Members**: A bracket on the right side of the code groups the four member declarations (`int age;`, `string first_name;`, `string last_name;`, and `int salary;`), with an arrow pointing to the label.

How to Define a Structure Variable in C++

- There are two ways to define a structure variable.

```
struct Person{  
    int age;  
    string first_name;  
    string last_name;  
    int salary;  
} p1, p2;
```

Both p1 and p2 are variable
of type Person.

```
int main(){  
  
    Person p1;  
  
    return 0;  
}
```

Here p1 is a variable
of type Person.

Member Functions in Structures

- Structures can also have member functions.
- These member functions are similar to regular functions but are defined within the scope of a structure. They can access and manipulate the data members of the structure directly.
- We can declare a member function by defining the function within the structure definition.

```
#include<iostream>
using namespace std;
struct Person{
    int age;
    string first_name;
    string last_name;
    int salary;
    void display(Person person){
        cout << "Name : " << person.first_name << " " << person.last_name << endl;
        cout << "Age : " << person.age << endl;
        cout << "Salary : " << person.salary << endl;
    }
}
p1, p2;
```

Member Functions in C++ }

Calling a member function.

```
int main(){
    // First Person
    Person p1;
    p1.age = 30;
    p1.first_name = "Muhammad";
    p1.last_name = "Awais";
    p1.salary = 30000;
    p1.display(p1);
    // Second Person
    Person p2;
    p2.first_name = "Kashif";
    p2.last_name = "Hameed";
    p2.age = 34;
    p2.salary = 50000;
    p2.display(p2);
    return 0;
}
```

Array of Structures

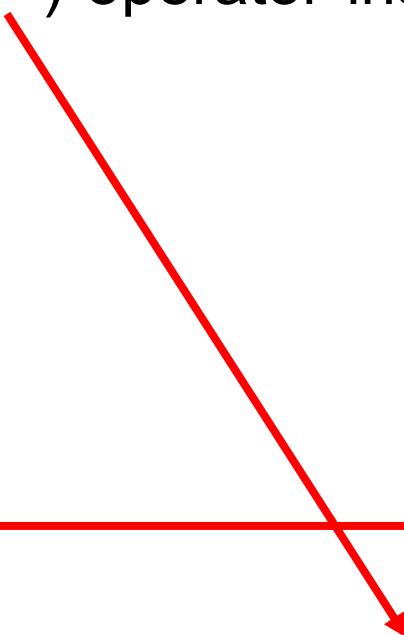
- Like other primitive data types, we can create an array of structures.

```
int main(){  
    // First Person  
    Person p1;  
    p1.age = 30;  
    p1.first_name = "Muhammad";  
    p1.last_name = "Awais";  
    p1.salary = 30000;  
    p1.display(p1);  
    // Second Person  
    Person p2;  
    p2.first_name = "Kashif";  
    p2.last_name = "Hameed";  
    p2.age = 34;  
    p2.salary = 50000;  
    p2.display(p2);  
  
    // Array of Structures  
    Person persons[2] = {p1,p2};  
    for (int i =0 ; i < 2 ; i++){  
        persons[i].print();  
    }  
    return 0;  
}
```

Structure Pointers

- Like primitive types, we can have pointer to a structure. If we have a pointer to structure, members are accessed using arrow (->) operator instead of the dot (.) operator.

```
int main(){
    Person p2;
    p2.first_name = "Kashif";
    p2.last_name = "Hameed";
    p2.age = 34;
    p2.salary = 50000;
    p2.display(p2);
    // Pointer to a structure
    Person *personPtr = &p2;
    cout << "Print name with pointer : " << personPtr->first_name << endl;
    return 0;
}
```



Tasks

- Declares a structure named Student to store:
 - First Name
 - Last Name
 - Roll Number
 - Marks
- Includes a member function displayStudentInfo() to:
 - Display the full name of the student.
 - Display the marks of the student.
- Uses a structure variable to:
 - Assign values to the data members.
 - Call the member function to display the student's information.

Tasks

- Repeat the previous program, but this time:
 - Use an array of structures to store details of multiple students.
 - Allow the user to input details for multiple students.
 - Call the displayStudentInfo() function for each student to display their full name and marks.
- Modify your previous program to:
 - Use a pointer to a structure to store and access student details.
 - Dynamically assign values to the structure members using the pointer.
 - Use the pointer to call the displayStudentInfo() function

Home Tasks

- What is the purpose of using **this** in member functions of structure?

References

- <https://www.geeksforgeeks.org/structures-in-cpp/>